# RACIOCÍNIO AUTOMÁTICO

Luís Morgado

ISEL-DEI

# PROCURA EM GRAFOS COM CICLOS

## ESTADOS REPETIDOS NA ÁRVORE DE PROCURA

- Grafo do espaço de estados apresenta ciclos
- Múltiplas transições para o mesmo estado
- Acções correspondentes às transições de estado são reversíveis

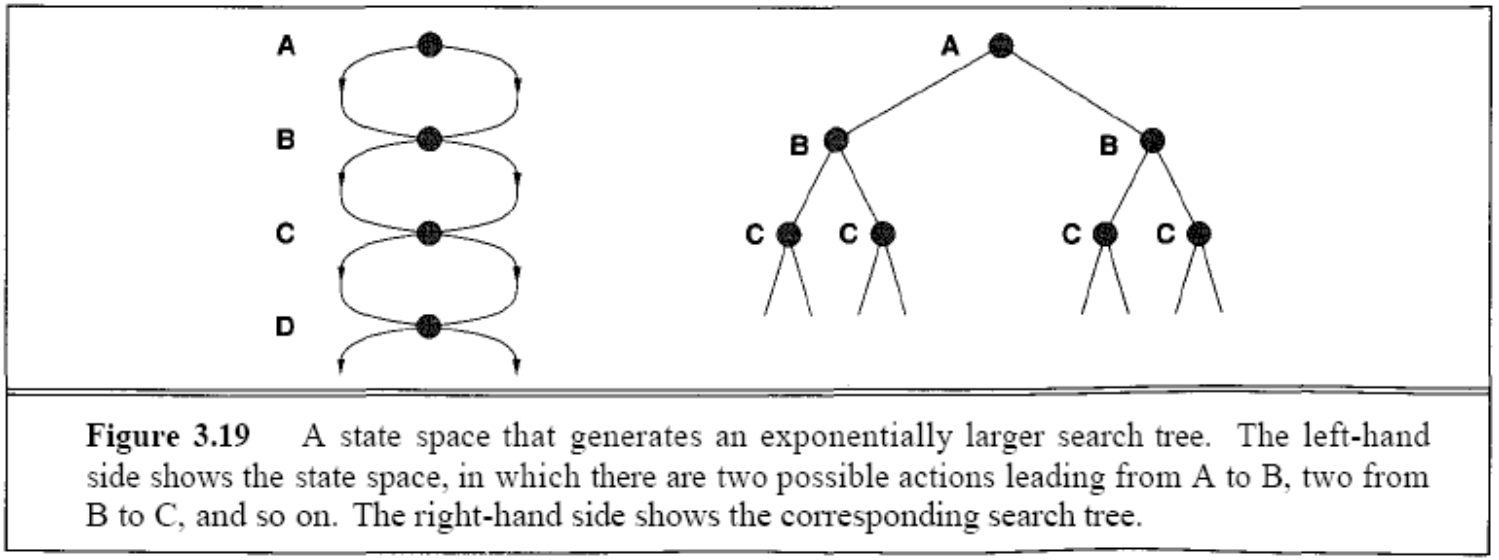



**Figure 3.19** A state space that generates an exponentially larger search tree. The left-hand side shows the state space, in which there are two possible actions leading from A to B, two from B to C, and so on. The right-hand side shows the corresponding search tree.

[Russel & Norvig, 2003]

## EXPANSÃO DE ESTADOS JÁ ANTERIORMENTE EXPLORADOS

- **Desperdício** de recursos (**tempo**, **memória**)

# PROCURA GERAL EM GRAFOS

- Para eliminação de nós correspondentes a estados repetidos, é necessário verificar se um novo *nó sucessor* corresponde a um estado que já foi anteriormente explorado
- Para esta verificação, é necessário ter em conta que os nós podem estar em duas situações distintas:
  - **Pendentes para expansão (*abertos*)**
    - Nós existentes na fronteira de exploração
  - **Expandidos (*fechados*)**
    - Nós anteriormente removidos da fronteira de exploração

# PROCURA EM GRAFOS COM CICLOS

## MEMÓRIA DE NÓS PROCESSADOS

- Nós gerados mas **não expandidos** (*fronteira de exploração*)
  - **ABERTOS**
- Nós gerados e **expandidos** por aplicação de operadores, gerando nós sucessores
  - **FECHADOS**

**EXPLORADOS**

# PROCURA GERAL EM GRAFOS

## No caso de existir uma métrica de custo associada a cada nó

- Para eliminação de nós correspondentes a estados repetidos, é necessário verificar se um novo *nó sucessor* corresponde a um estado que já foi anteriormente explorado
  - Se isso acontecer, apenas o nó que corresponde ao percurso com **menor custo** deve **ser mantido**
  - O outro nó correspondente ao mesmo estado, mas num percurso com **maior custo** deve ser **eliminado**

# RACIOCÍNO ATRAVÉS DE PROCURA

## Complexidade computacional

### FACTOR DE RAMIFICAÇÃO

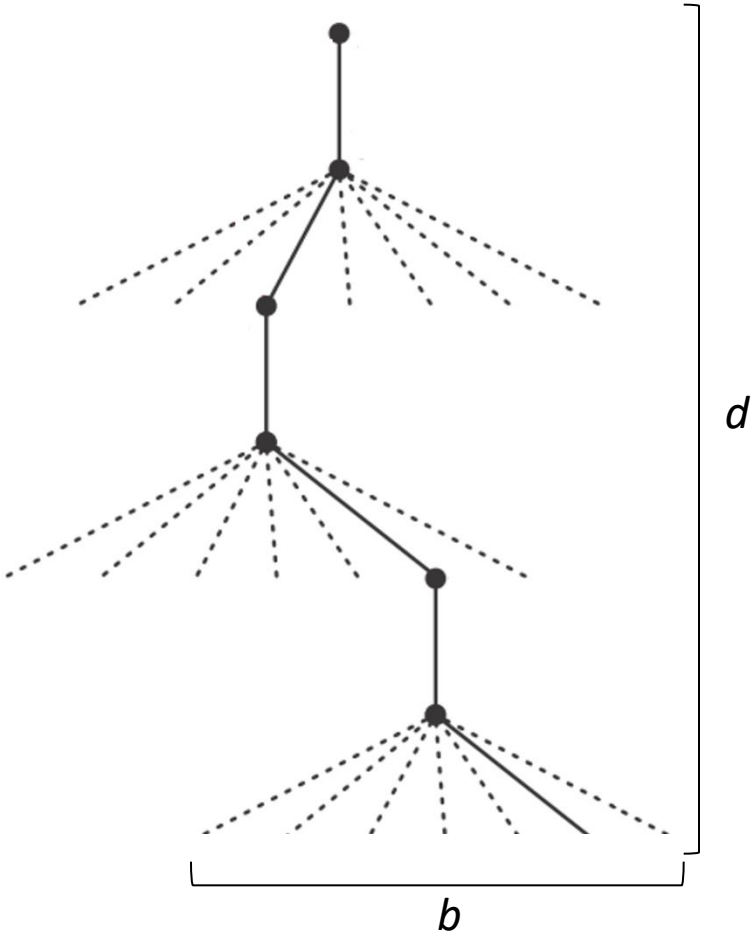
***b* –** *branching factor*

- Número máximo de sucessores para um qualquer estado

### PROFUNDIDADE DA PROCURA

***d* –** *depth*

- Dimensão do percurso entre o estado inicial e o estado objectivo




**Evolução exponencial do número *N* de percursos possíveis**

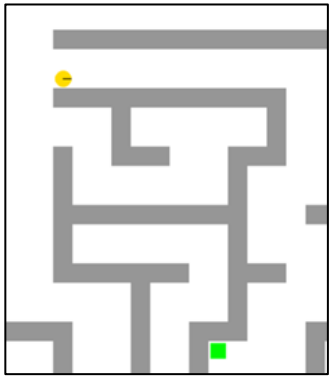
$$N = \underbrace{b \times b \times \ldots b}_{d} = b^d$$

**Explosão combinatória**

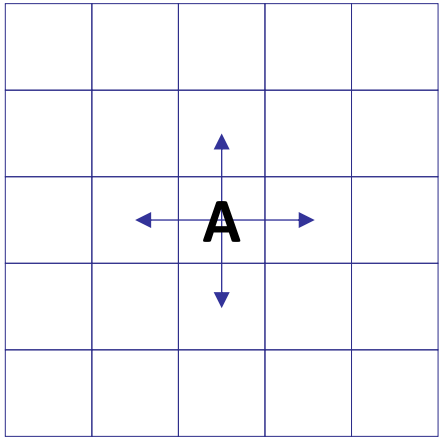
# RACIOCÍNO ATRAVÉS DE PROCURA
## Complexidade computacional

Consideremos um contexto simplificado em que o sistema se movimenta em passos discretos em quatro direcções possíveis

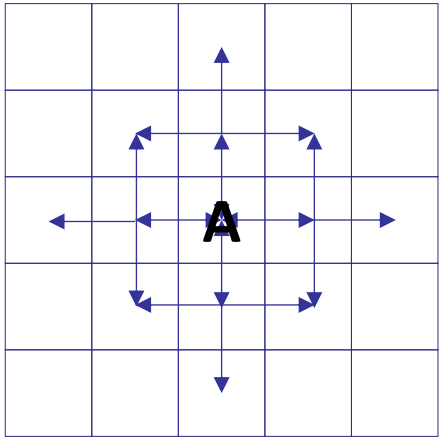
**Quantos percursos possíveis existem para atingir um alvo?**



Um movimento




Dois movimentos

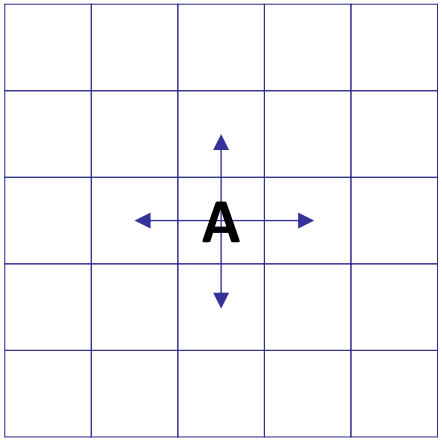


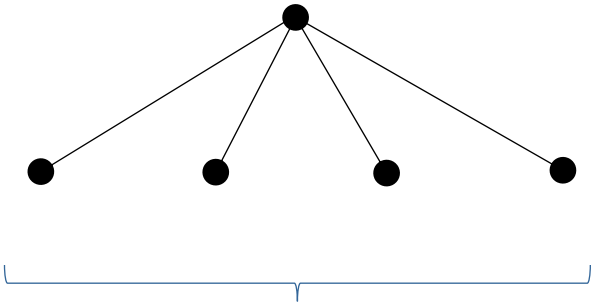

Modelação do problema

- **Estado**: posição (X, Y) do agente
- **Acção**: movimento numa das quatro direcções

# Árvore de procura

Um movimento



***b***

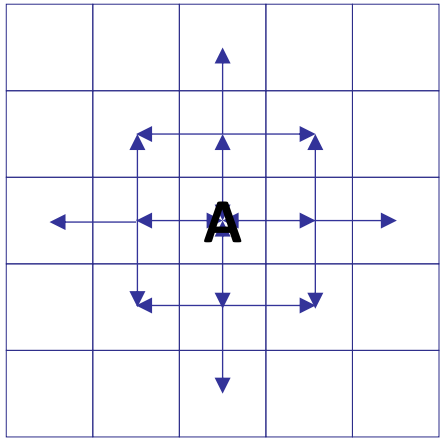
Factor de ramificação
(*branching factor*)

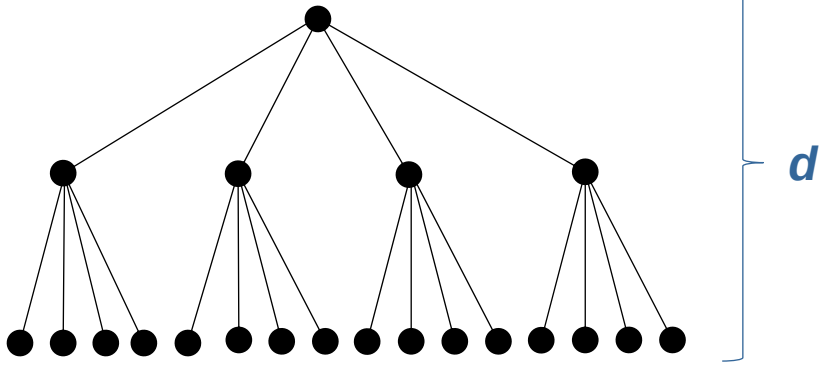
***b*** = 4

***d*** = 1

Nós processados: 1 + 4

Dois movimentos



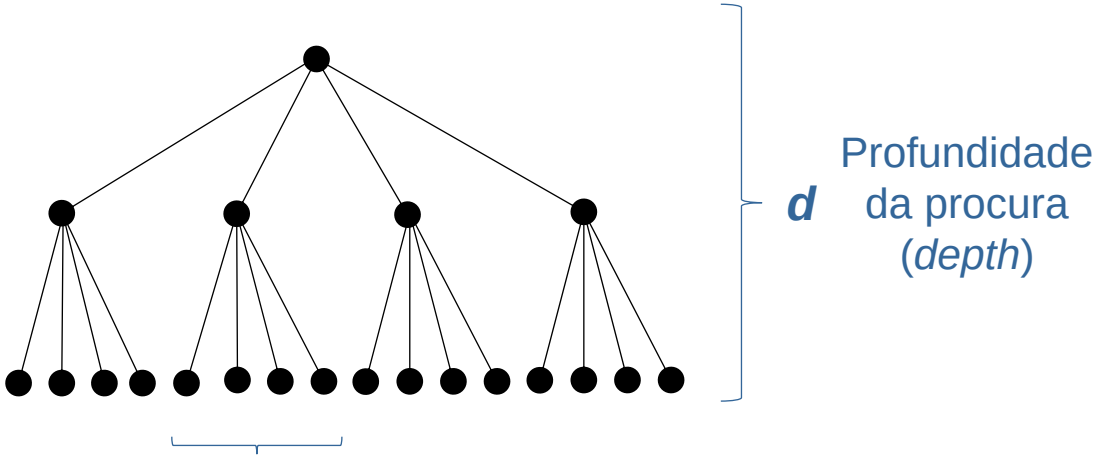




Profundidade da procura
(*depth*)

***d*** = 2

Nós processados: 1 + 4 + 16

# Árvore de procura




***b*** = 4

Nós processados: 1 + 4 + 16 + 64 + …

No caso geral, número de nós a expandir para encontrar uma solução de profundidade ***d***

$1 + b + b^2 + b^3 + \ldots + b^d$

**Complexidade temporal**: $O(b^d)$

**Notação $f = O(g)$**

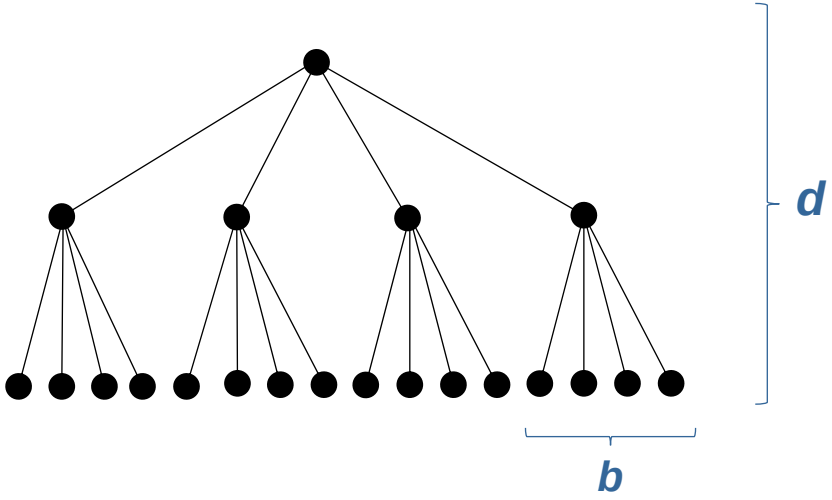
$f(x)$ é de ordem $O(g(x))$ se $f(x) \leq cg(x)$ para $x > x_0$

$x_0$, $c$: constantes positivas

**COMPLEXIDADE TEMPORAL EXPONENCIAL**

# Complexidade espacial (memória necessária)

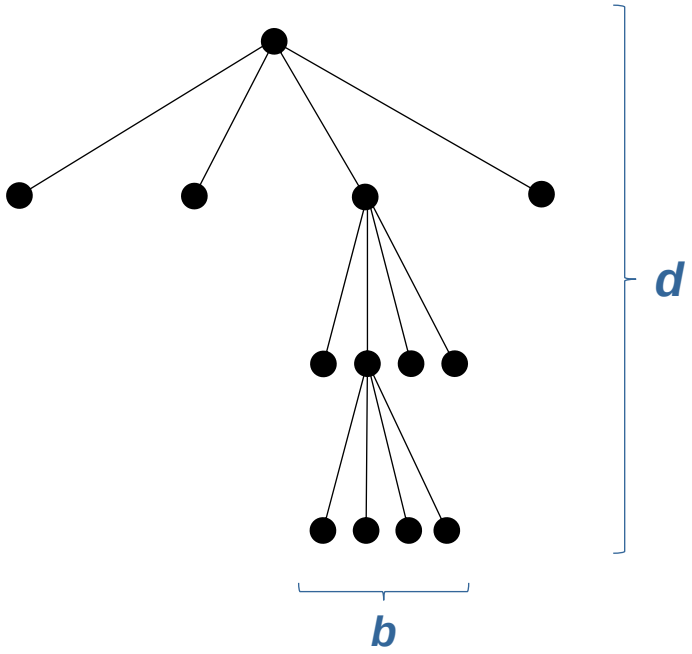
## Procura em largura




Complexidade espacial: $O(b^d)$

**Exponencial**

## Procura em profundidade




Complexidade espacial: $O(bd)$

**Linear**

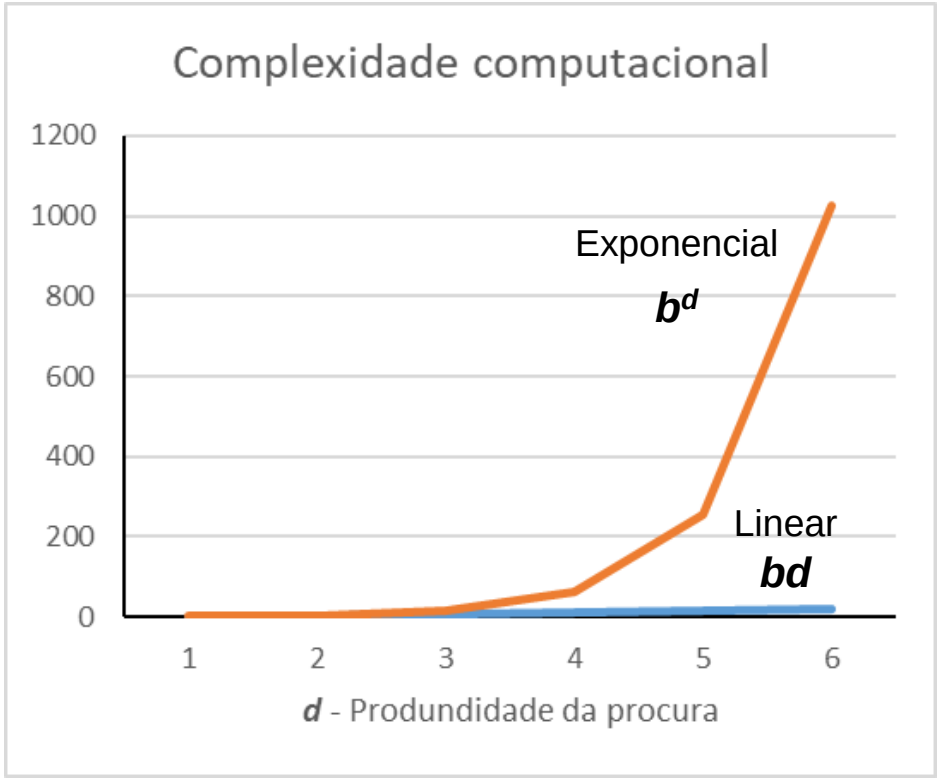
**No entanto:**

- **Pode não encontrar solução**
- **Solução pode não ser óptima**

# RACIOCÍNO ATRAVÉS DE PROCURA
## Complexidade computacional

Complexidade computacional de um método de procura:

- **Factor de ramificação** - ***b***
  - Número máximo de sucessores para um qualquer estado
- **Profundidade da procura** - ***d***
  - Profundidade do nó objectivo menos profundo na árvore de procura
  - Dimensão do percurso entre o estado inicial e o estado objectivo



# RACIOCÍNO ATRAVÉS DE PROCURA

- **Aspectos a considerar num método de procura**
  - **Completo**
    - O método de procura garante que, caso exista solução, esta será encontrada
  - **Óptimo**
    - O método de procura garante que, existindo várias soluções, a solução encontrada é a melhor
  - **Complexidade**
    - **Tempo** (complexidade temporal)
      - **Tempo necessário** para encontrar uma solução
    - **Espaço** (complexidade espacial)
      - **Memória necessária** para encontrar uma solução

# COMPLEXIDADE COMPUTACIONAL

| Método de Procura | Tempo | Espaço | Óptimo | Completo |
|---|---|---|---|---|
| **Profundidade** | $O(b^m)$ | $O(bm)$ | Não | Não |
| **Largura** | $O(b^d)$ | $O(b^d)$ | Sim | Sim |
| **Custo Uniforme** | $O(b^{[C^*/\varepsilon]})$ | $O(b^{[C^*/\varepsilon]})$ | Sim | Sim |
| **Profundidade Limitada** | $O(b^l)$ | $O(bl)$ | Não | Não |
| **Profundidade Iterativa** | $O(b^d)$ | $O(bd)$ | Sim | Sim |

$b$ – factor de ramificação
$d$ – dimensão da solução
$m$ – profundidade da árvore de procura
$l$ – limite de profundidade

$C^*$ – Custo da solução óptima
$\varepsilon$ – Custo mínimo de uma transição de estado ($\varepsilon > 0$)

**Notação *O***

$g(n)$ é de ordem $O(f(n))$ se existirem duas constantes positivas $k$ e $N$ tal que:

$\forall (n > N) : g(n) \leq kf(n)$

# BIBLIOGRAFIA


[Russel & Norvig, 2010]

S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd Edition, Prentice Hall, 2010

[Nilsson, 1998]

N. Nilsson , *Artificial Intelligence: A New Synthesis*, Morgan Kaufmann 1998

[Luger, 2009]

G. Luger , *Artificial Intelligence: Structures and Strategies for Complex Problem Solving* , Addison-Wesley, 2009

[Jaeger & Hamprecht, 2010]

M. Jaeger, F. Hamprecht, *Automatic Process Control for Laser Welding*, Heidelberg Collaboratory for Image Processing (HCI) , 2000

[Pearl, 1984]

J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, 1984

[Montemerlo, 2008]

M. Montemerlo *et al.*, *Junior: The Stanford Entry in the Urban Challenge*, Stanford Artificial Intelligence Lab, 2008

[Nagy et al., 2018]

D. Nagy, D. Zhao, D. Benjamin, “Nature-Based Hybrid Computational Geometry System for Optimizing Component Structure”, Humanizing Digital Reality, Springer Nature, 2018